



PAUTA TAREA 4

Pregunta 1

Pregunta 1.1

Una posible solución consiste en transformar las expresiones P y R en autómatas $\mathcal{A}_P = (Q_P, \Sigma, \Delta_P, I_P, F_P)$ y $\mathcal{A}_R = (Q_R, \Sigma, \Delta_R, q_0, F_R)$. Luego se ejecuta $\mathcal{A}_P$ sobre el documento, marcando las posiciones en las que $\mathcal{A}_P$ llega a un estado final. Después basta modificar el algoritmo `evalLeftmostLongest` para que solo se agregue una ejecución en la posición i si la posición $i - 1$ estaba marcada como final para $\mathcal{A}_P$. Esto mantiene la complejidad lineal entre outputs del algoritmo `evalLeftmostLongest`. El algoritmo en pseudo-código resulta:

```
function EVALLEFTMOSTLONGESTPR( $\mathcal{A}_P, \mathcal{A}_R, w$ )
   $n := w.length$ 
   $M := [\text{false}; n + 1]$ 
  if  $I_P \cap F_P \neq \emptyset$  then  $M[0] := \text{true}$ 
   $P' = I_P$ 
  for  $i = 1$  to  $n$  do
     $P_t := \{q \in Q_P \mid p \in P' \wedge (p, a_i, q) \in \Delta_P\}$ 
     $P' := P_t$ 
    if  $P' \cap F_P \neq \emptyset$  then
       $M[i] = \text{true}$ 
   $P := \emptyset, s := \text{null}$ 
  for  $i = 1$  to  $n$  do
    if  $s = \text{null}$  and  $M[i - 1]$  then
       $P := P \cup \{(q_0, i)\}$ 
       $P := \text{updateSet}(\Delta_R, a_i, P)$ 
    if  $\{p \mid (p, k) \in P\} \cap F_R \neq \emptyset$  then
       $m := \min\{k \mid (p, k) \in P \wedge p \in F_R\}$ 
      if  $s := \text{null} \vee m \leq s.start$  then
         $s := [m, i + 1)$ 
    if  $(P \neq \emptyset \vee i = n) \wedge s \neq \text{null}$  then
      output  $s$ 
       $i := s.end, P := \emptyset, s := \text{null}$ 
```

▷ Arreglo de largo $n + 1$ indexado desde 0

Dado lo anterior, el puntaje asignado es el siguiente:

- (4 puntos) Por entregar un algoritmo correcto que funcione en tiempo lineal entre outputs.
- (3 puntos) Por entregar un algoritmo correcto que funcione en tiempo cuadrático entre outputs.
- (0 puntos) En otro caso.

Pregunta 1.2

De manera análoga al algoritmo anterior, una posible solución consiste en transformar las expresiones R y S en autómatas $\mathcal{A}_R = (Q_R, \Sigma, \Delta_R, q_0, F_R)$ y $\mathcal{A}_S = (Q_S, \Sigma, \Delta_S, I_S, F_S)$ y luego *dar vuelta* el automata $\mathcal{A}_S$, intercambiando estados finales e iniciales y dando vuelta las transiciones. Luego se ejecuta el autómata dado vuelta sobre el documento invertido, marcando las posiciones donde el autómata llega a un estado final. La idea es que si la posición i es marcada como final para el autómata invertido, entonces el autómata normal acepta $w_i \dots w_n$.

Luego basta modificar el algoritmo `evalLeftmostLongest` para que solo revise los estados finales de $\mathcal{A}_R$ si la posición $i + 1$ está marcada. Al igual que antes, esto mantiene la complejidad lineal entre outputs del algoritmo `evalLeftmostLongest`. El algoritmo en pseudo-código resulta:

```

function EVALLEFTMOSTLONGESTRS( $\mathcal{A}_R, \mathcal{A}_S, w$ )
   $n := w.length$ 
   $M := [\text{false}; n + 2]$  ▷ Arreglo de largo  $n + 2$  indexado desde 0
  if  $I_S \cap F_S \neq \emptyset$  then  $M[n + 1] := \text{true}$ 
   $P' = F_S$  ▷ Se trabaja sobre  $\mathcal{A}_S$  invertido de manera implícita
  for  $i = n$  downto 1 do
     $P_t := \{p \in Q_P \mid q \in P' \wedge (p, a_i, q) \in \Delta_S\}$ 
     $P' := P_t$ 
    if  $P' \cap I_S \neq \emptyset$  then
       $M[i] = \text{true}$ 
   $P := \emptyset, s := \text{null}$ 
  for  $i = 1$  to  $n$  do
    if  $s = \text{null}$  then
       $P := P \cup \{(q_0, i)\}$ 
       $P := \text{updateSet}(\Delta_R, a_i, P)$ 
    if  $M[i + 1] \wedge \{p \mid (p, k) \in P\} \cap F_R \neq \emptyset$  then
       $m := \min\{k \mid (p, k) \in P \wedge p \in F_R\}$ 
      if  $s := \text{null} \vee m \leq s.start$  then
         $s := [m, i + 1]$ 
    if  $(P \neq \emptyset \vee i = n) \wedge s \neq \text{null}$  then
      output  $s$ 
       $i := s.end, P := \emptyset, s := \text{null}$ 

```

Dado lo anterior, el puntaje asignado es el siguiente:

- (4 puntos) Por entregar un algoritmo correcto que funcione en tiempo lineal entre outputs.
- (3 puntos) Por entregar un algoritmo correcto que funcione en tiempo cuadrático entre outputs.
- (0 puntos) En otro caso

Pregunta 2

Pregunta 2.1

Una posible solución consistía en demostrar que todos los estados del autómata $\mathcal{A}_w^{\text{lazy}}$ son distinguibles. Una forma de hacer esto es que, sean $w = a_0 \cdot \dots \cdot a_{n-1}$ una palabra y $\mathcal{A}_w^{\text{lazy}}$ la determinización de $\mathcal{A}_w$, se puede mostrar que cada estado S_i de $\mathcal{A}_w^{\text{lazy}}$ llega a un estado final con la palabra $w_i = a_i \cdot \dots \cdot a_{n-1}$ y que ningún otro estado S_j con $j < i$ llega a un estado final con la misma palabra. Por lo tanto, todos los estados pueden ser distinguidos de los demás y por lo tanto el autómata es mínimo.

Dado lo anterior, el puntaje asignado es el siguiente:

- **(4 puntos)** Por demostrar correctamente que cada estado del autómata $\mathcal{A}_w^{\text{lazy}}$ es distinguible.
- **(3 puntos)** Por tener un error desmostrando que cada estado es distinguible (por ejemplo, decir que basta que $j \neq i$).
- **(0 puntos)** En otro caso.

Pregunta 2.2

El truco para hacer el algoritmo en $\mathcal{O}(|w|)$ es basarse en que cada estado tiene solo dos transiciones, la que avanza y la que se queda en la misma posición (excepto en el caso del primer estado, donde ambas avanzan). Las transiciones que leen la palabra son triviales, por lo que solo se entregará la intuición de como hacer las transiciones que no consumen letras. Para esto, lo primero que se debe notar es que si el estado anterior al actual se devuelve al estado S_0 , entonces el nodo actual solo puede volver a S_0 o S_1 . Por otro lado, si es que el nodo anterior salta hacia un nodo S_i con $i \neq 0$, entonces se debe buscar iterativamente hacia atrás de la siguiente forma. Sean S_j el nodo actual, S_{j-1} su nodo anterior y $S = S_i$ el nodo al que se devuelve el estado S_{j-1} .

1. Si la letra para pasar de S a su siguiente estado es la misma que la letra entre S_{j-1} y S_j , retornar el siguiente estado desde S .
2. Si $S = S_0$ retornar S_0 .
3. En otro caso, setear S al estado al que se devuelve el nodo S y volver al paso 1.

Si bien pareciera que lo anterior hace el algoritmo cuadrático, es posible demostrar que sumando todas las veces que se ejecuta la iteración luego de leer la palabra completa solo llega a $\mathcal{O}(|w|)$ en el peor caso, por lo que no aumenta la complejidad pedida. La demostración de esto queda propuesta.

Dado lo anterior, el puntaje asignado es el siguiente:

- **(4 puntos)** Por entregar un algoritmo correcto con la complejidad pedida.
- **(3 puntos)** Por tener errores menores en el algoritmo (por ejemplo, fallar en casos muy específicos).
- **(0 puntos)** En otro caso.